



Arbeitsweise der Transportschicht



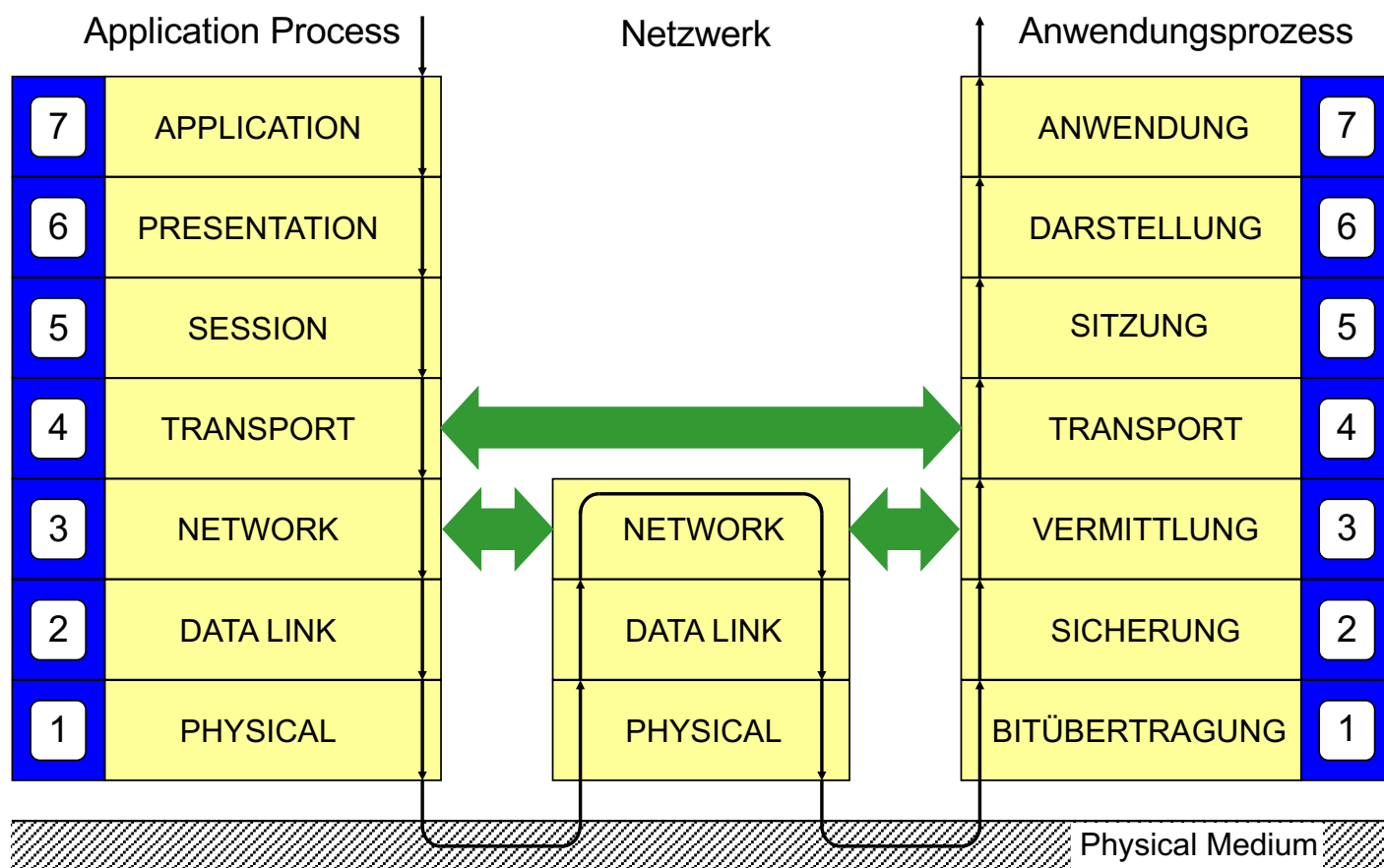
Transport- und Vermittlungsschicht

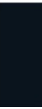
- Vermittlungsschicht: logische Kommunikation zwischen *Rechnern in einem Netz aus Netzen*
 - Anbindung von Rechnern (eigentlich sogar Netzwerkkarten) in die Netze
 - Regelung des netzübergreifenden Verkehrs:
 - > Kopplung von lokalen Netzen
 - > Globale Adressierung
 - > Routing von Datenpaketen
- Transportschicht: logische Kommunikation **zwischen Prozessen**
 - Adressierung der Anwendungen, Segmentierung der Datenströme
 - Benutzt und erweitert die Dienste der Vermittlungsschicht
 - Das erzielbare Dienstangebot wird daher von den Fähigkeiten der Vermittlungsschicht beeinflusst!





Die Transportschicht ist eine Endsystemschiicht





Aufgrund der besonderen Struktur (Store-and-Forward, kooperierende Autonome Systeme) gilt:

Im Internet kann eine zuverlässige Kommunikation zwischen Prozessen nur auf Transport- oder auf Anwendungsebene realisiert werden!



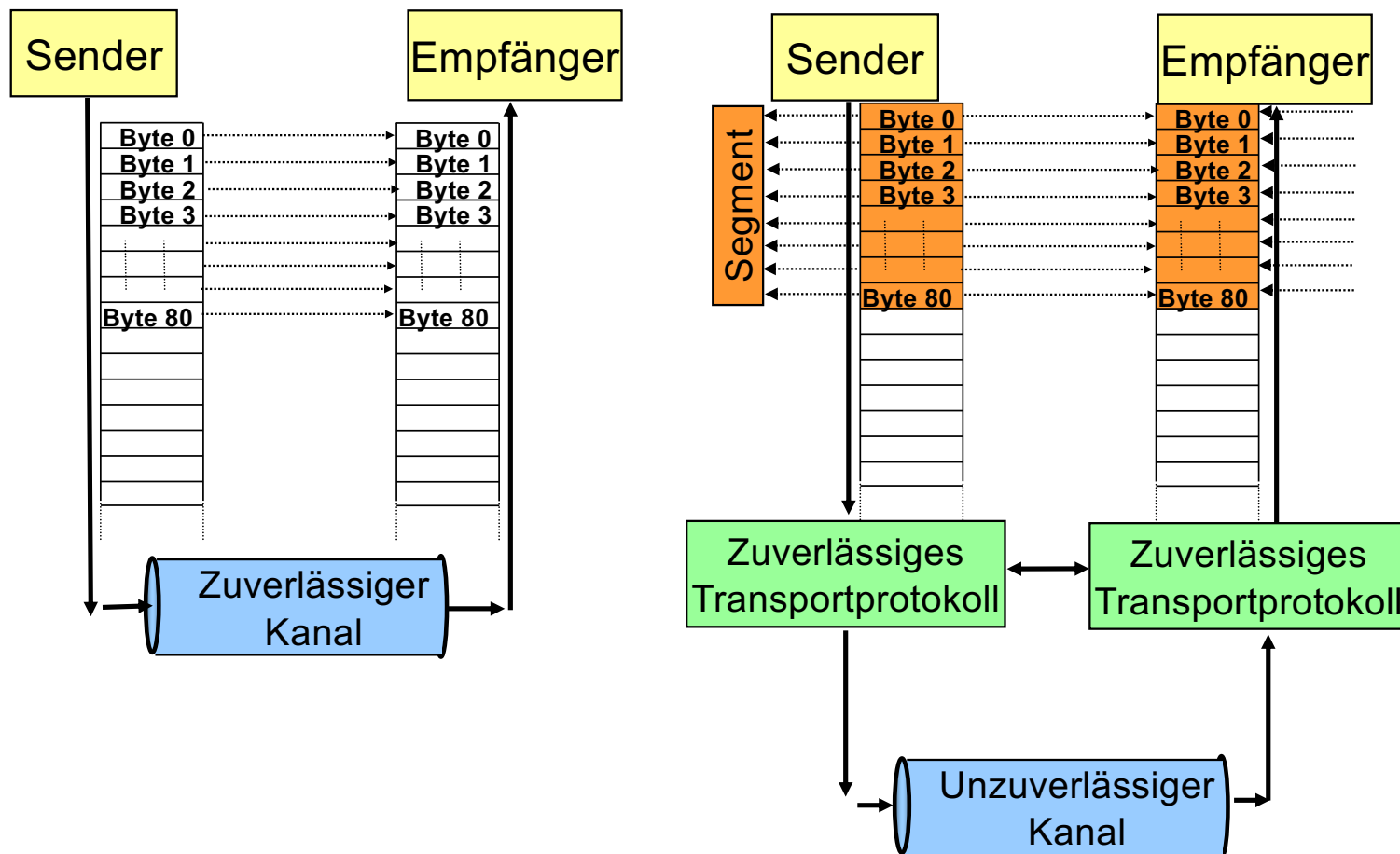
Internet-Protokolle – TCP und UDP

- Transportprotokolle im Internet (im Betriebssystem implementiert)
 - Verbindungsorientierte Kommunikation
 - Verbindungslose Kommunikation
- Transmission Control Protocol (TCP) -> zuverlässig
 - Kommunikationsprimitive, Sockets, Ports
 - **Virtuelle Verbindungsorientierung**: Modell zweier Streams, die die Prozesse miteinander verbinden; „Open“-“Close“ der Streams sind die virtuellen Verbindungen
 - Flusskontrolle, Staukontrolle
- User Datagram Protocol (UDP) -> unzuverlässig
 - Kommunikationsprimitive, Sockets, Ports
 - Postkartenmodell: Ein Empfangspunkt (Port) für alle Nachrichten, versenden ohne Absprache mit dem Empfänger





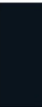
Zuverlässiges Transportprotokoll





Ausgangspunkt

- Wie können eigentlich zuverlässige Protokolle über dem unsicheren IP implementiert werden?
 - Pakete können verloren gehen
 - Die Reihenfolge der Pakete kann sich ändern
 - Pakete können fehlerhaft empfangen werden
- Anmerkung
 - Bei der folgenden Darstellung gehen wir von einer existierenden „Verbindung“ zwischen Sender und Empfänger aus
 - Sender und Empfänger haben also bereits Nachrichten ausgetauscht und wissen voneinander





Zuverlässige Transportprotokolle

- Erforderliche Mechanismen

1. Aufteilen des Byte-Streams in Nachrichteneinheiten (**Segmente**)
2. Erkennen der **Position** einer empfangenen Nachricht im logischen Byte-Stream - Nummerierung der Segmente (**Sequenznummern**)
3. Empfänger-Feedback
 - > **Quittierungsnachrichten** zwischen Empfänger und Sender
 - > Positive **ACK**nowledgment (**ACK**)
 - Empfänger bestätigt den korrekten Erhalt eines Segments mit einem ACK
 - Hierbei sollte der Empfänger vorher auf Bitfehler geprüft haben
 - > **Optional: Negative ACK**nowledgment (**NAK**)
 - Empfänger informiert Sender über „Lücken“ oder Fehler
 - Ursprungslänge der Nachricht muss abgeleitet werden können

4. Neuübertragung

- > Sender überträgt Nachrichten (Segmente) erneut
- > Timer-gesteuert: Wenn bis zum Ablauf des Timers keine Bestätigung erhalten wurde, dann wird das Segment erneut übertragen
- > **Optional:** Bei Erhalt eines NAK



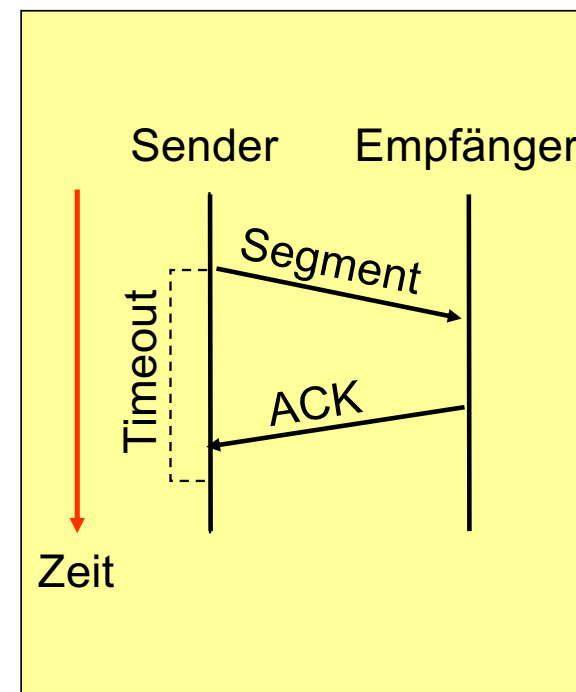
Das “Stop and Wait”-Protokoll

■ Automatic Repeat-reQuest (ARQ)

- Empfänger verschickt Bestätigungen (**ACKs**) bei korrektem Empfang
- Fehlerhafte Übertragungen werden durch ausbleiben der Bestätigung erkannt und vom Sender durch erneute Übertragung behoben

■ Einfachstes ARQ-Protokoll

- Segmentweises Vorgehen
- Timer zur Fehleridentifikation
 - Verschicke ein Segment und warte auf den Empfang des ACKs
 - Bei Empfang eines ACKs wird das nächste Segment verschickt
 - Springt der Timer an, so wird die Übertragung wiederholt





Schlechter Nutzungsgrad der Verbindungskapazität

- Jeder Datenstrom wird in Segmente unterteilt, z.B. 1460 Bytes Nutzdaten
- Nach jedem gesendeten Segment wird auf eine Bestätigung gewartet; während der **Wartezeit** werden somit keine weiteren Segmente übertragen
 - ➔ Begrenzung auf **ein Paket pro Zyklus (Round-Trip)!!!!**
- **Nutzungsgrad:** Verhältnis von genutzter Übertragungszeit zu gesamter Übertragungsdauer (inkl. der Wartezeit)

$$\rho = (\text{genutzte Übertragungszeit}) / (\text{genutzte Übertragungszeit} + \text{Wartezeit})$$

Sei L die Länge eines Segmentes

Sei R die Übertragungsrate des Kanals

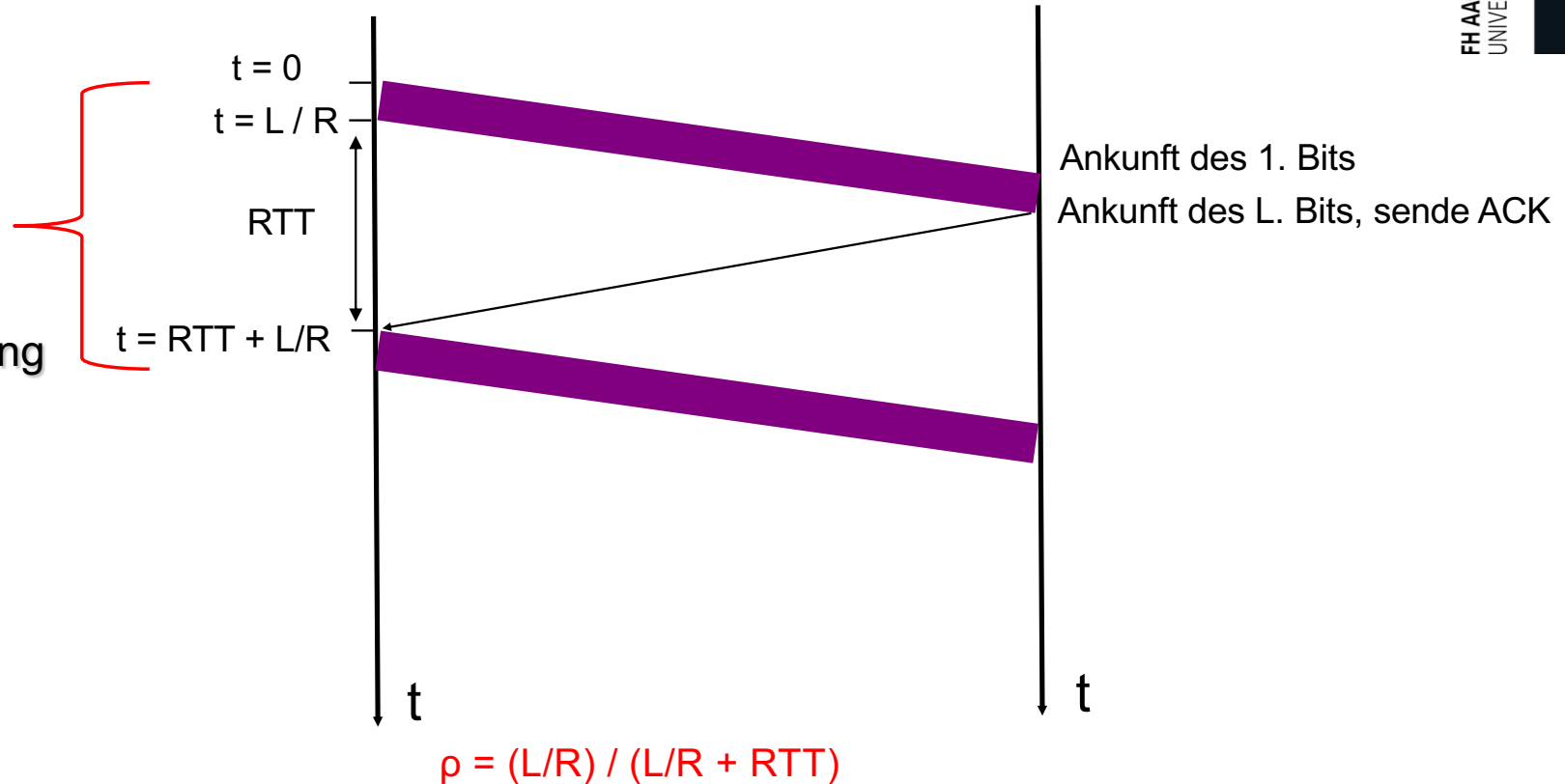




Schlechter Nutzungsgrad der Verbindungskapazität

Hinweis:

Tatsächlich startet die Round-Trip-Time ab dem Senden, also aber dem Übertragen des ersten Bytes und sie endet mit dem vollständigen Empfang der Antwort





Schlechter Nutzungsgrad der Verbindungskapazität

Beispiel: FastEthernet im LAN

$R = 100 \text{ Mbit/s}$

$L = 1460 \text{ Byte} \quad L/R = 0,0001168$

$RTT = 3 \text{ ms}$

$$\rho = (L/R) / (L/R + RTT) = 0,037474 = 3,74\%$$

Beispiel: Kommunikation mit Kalifornien

$R = 100 \text{ Mbit/s}$

$L = 1460 \text{ Byte} \quad L/R = 0,0001168$

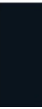
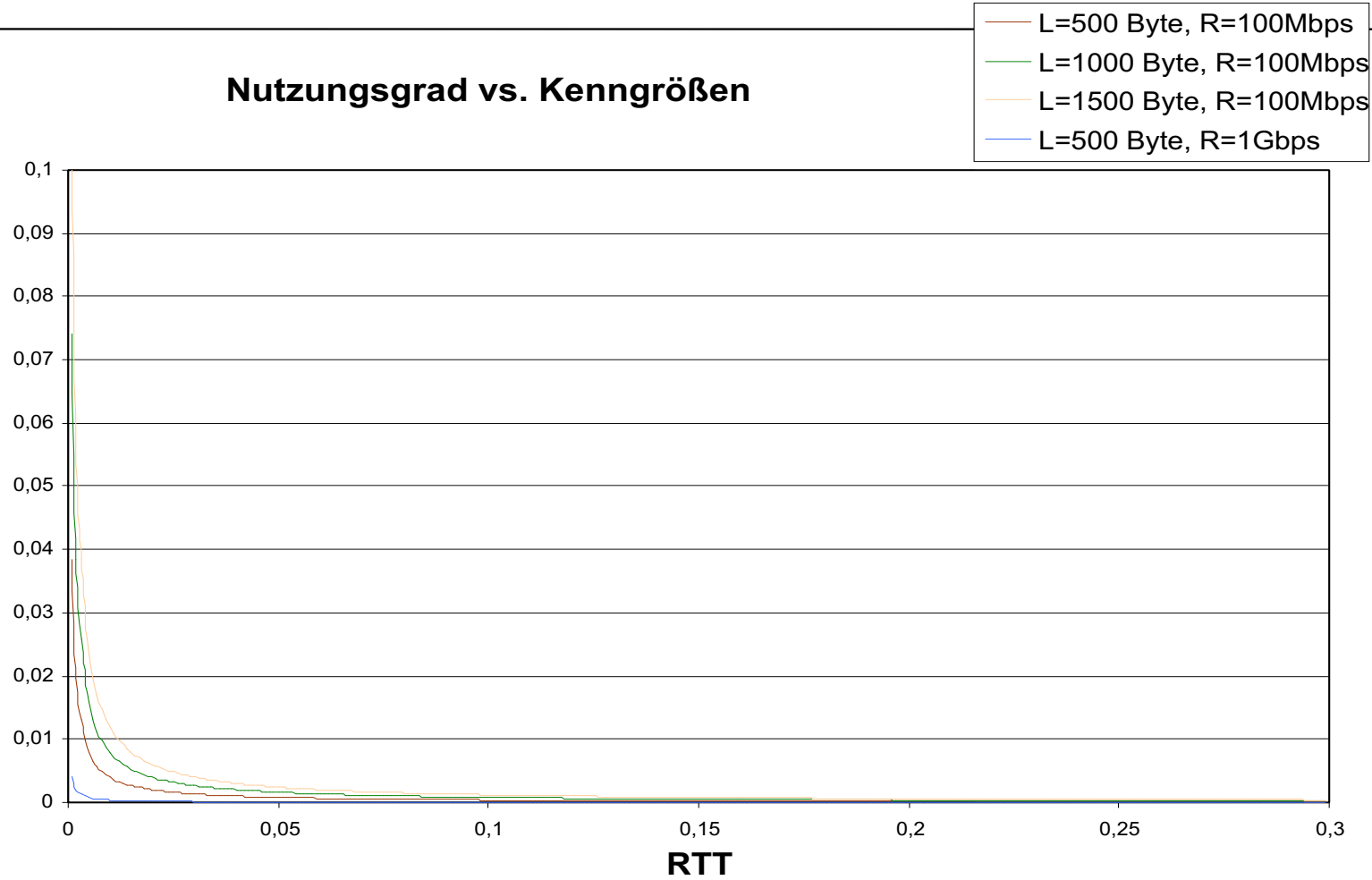
$RTT = 200 \text{ ms} \quad \rho = (L/R) / (L/R + RTT) = 0,000584 = 0,058\%$





Schlechter Nutzungsgrad der Verbindungskapazität

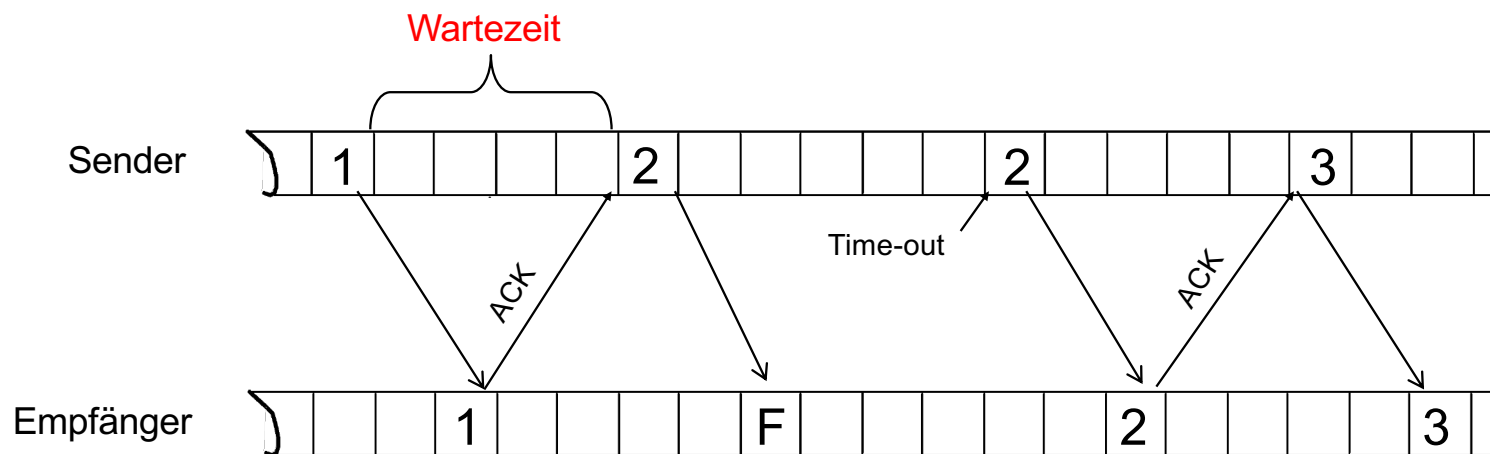
Nutzungsgrad vs. Kenngrößen





Eins nach dem Anderen: **Stop-and-Wait**

- Sender überträgt ein Segment und startet einen Timer
 - Trifft eine Quittung ein: sende das nächste Segment
 - Tritt ein Timeout ein (d.h. läuft der Timer ab): wiederhole das vorherige Segment
- Lange Wartezeiten zwischen den Segmenten! Dadurch Verschwendung von Übertragungskapazität!





Verfahren bei Fehler/Sonderfällen

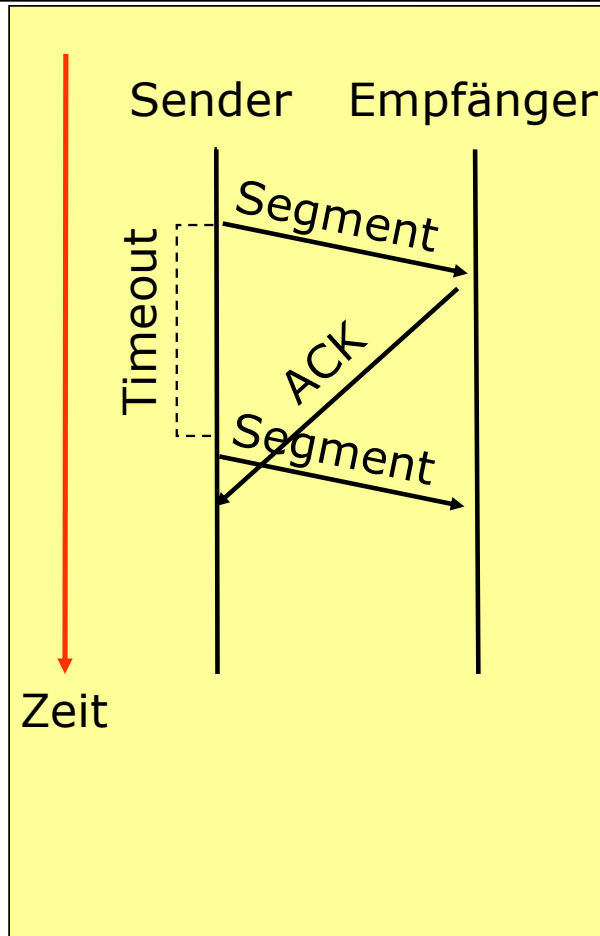


Welche Ereignisse können den geschilderten
Ablauf stören?



Verfahren bei Fehler/Sonderfällen

Welche Bestätigung
haben wir hier
empfangen?



Welches Segment
haben wir hier
empfangen?



Konsequenz

- Wir benötigen eine **eindeutige Identifikation der Nachrichten**
- Damit müssen wir die **Segmente und** die dazugehörigen **Bestätigungen** mit einer eindeutigen Nummer versehen

Lösungsansätze:

- Wir verwenden den **Byte-Offset** im Stream in der Art, dass für das aktuelle Segment angezeigt wird, in welcher Position das 1. Byte liegt
Alternative: Wir **nummerieren** die Nachrichten durch
- Die Nummerierung sollte zyklisch sein, da der Zahlenraum bei Protokollen begrenzt ist
- Bestätigungen verweisen entweder auf **das nächste erwartete Byte/Nummer** oder auf das zuletzt empfangene



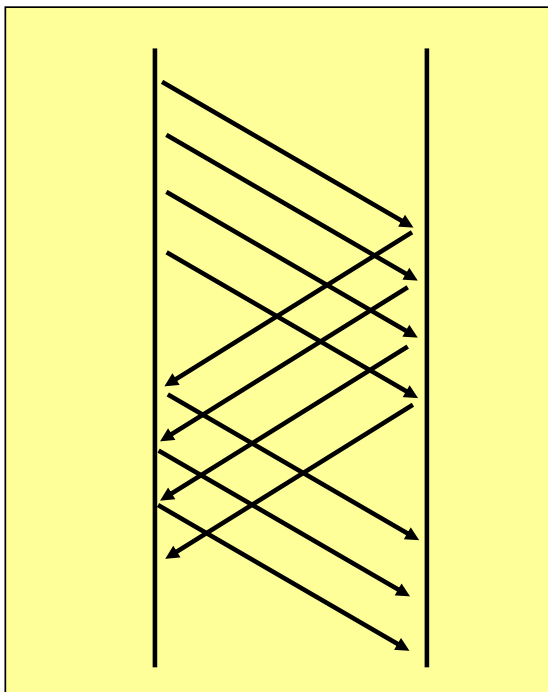


Idee: Segmentnummern können doch auch für Bestätigungen verwendet werden. Die Segmente und die Bestätigungen werden also einfach nummeriert. Eine Bestätigung zeigt so entweder an, welche Segmentnummer gerade empfangen wurde oder welche als nächstes erwartet wird.

In der Realität wird zumeist der Byte-Offset im Stream verwendet.

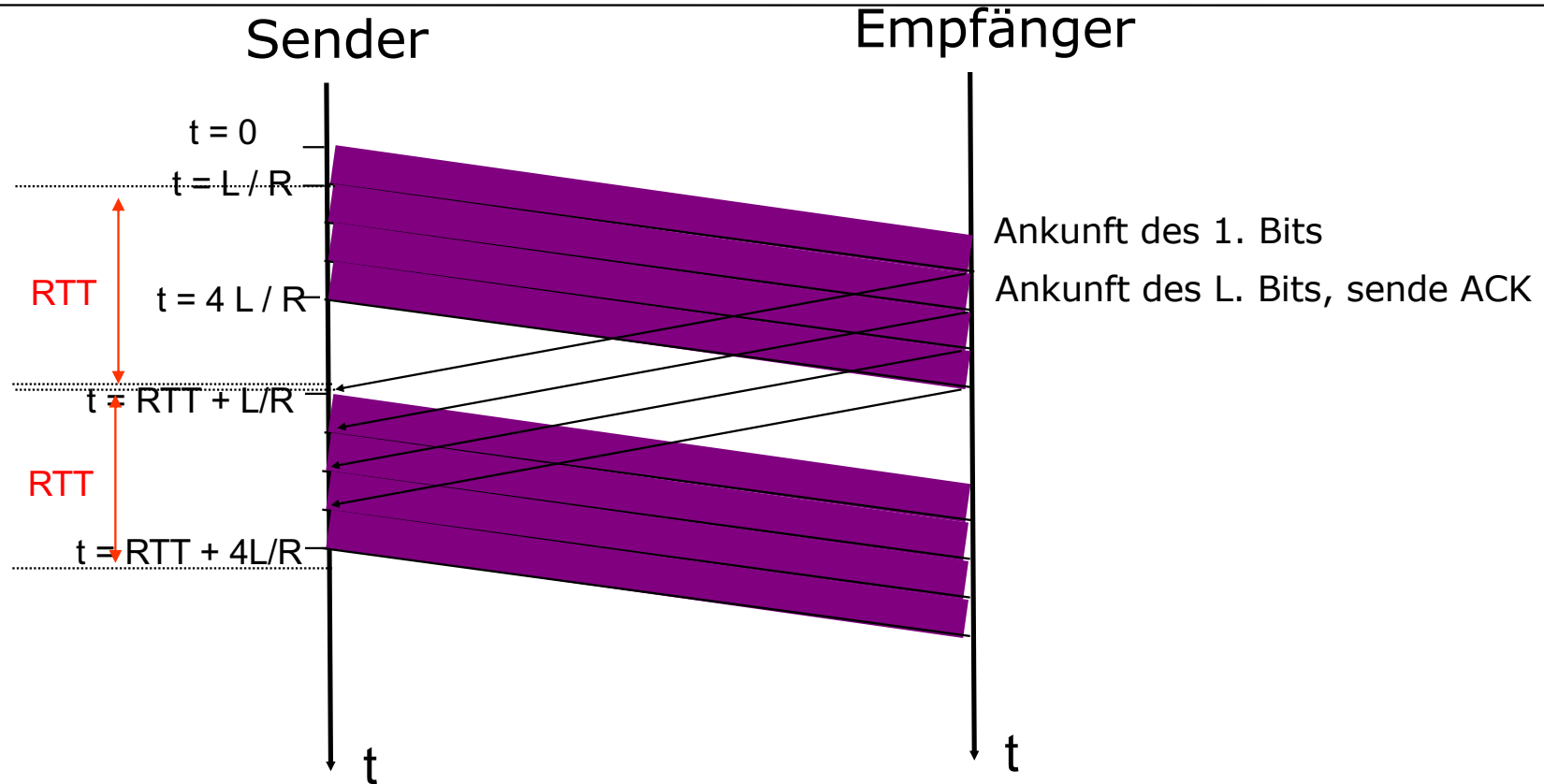


Wie kann man das Netz besser auslasten?





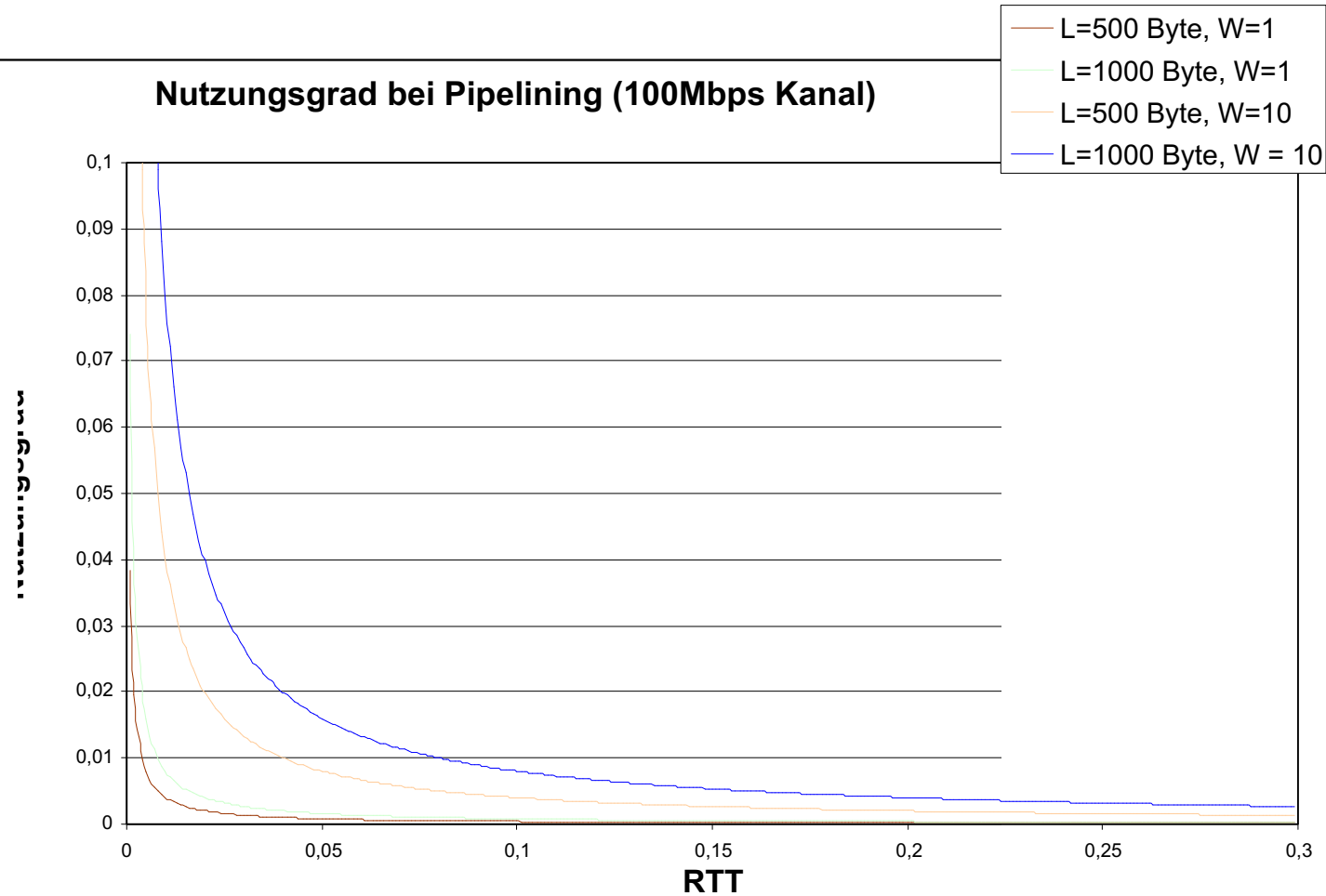
Flusskontrolle folgt dem Pipelining-Prinzip



$$\rho = (4L/R) / (4L/R + (RTT + L/R - 4L/R)) = (4L/R) / (L/R + RTT)$$



Schlechter Nutzungsgrad der Verbindungskapazität





Wir erhöhen so aber die Komplexität der Behandlung von Fehlern

Einfachste Lösung: Go-Back-N



Go-Back-N

